Reinforcement Learning for Pseudo-Labeling

Capstone Project Presentation

Group-11

Students: Aishwarya Maddula, Deepika Reddygari, Phanindra Kalaga, Prudhvi Chekuri

Guided by: Dr. Amir Jafari, Prof. Tyler Wallett

George Washington University

Outline

1. Introduction
2. Background and Literature Review
3. Reinforcement Learning Framework
4. Project Setup
5. PseudoCode
6. Implementation Details
7. Results and Insights
8. PseudoCode RLGSSL
9. Results and Insights RLGSSL
10. Conclusion
11. Limitations
12. Future Scope

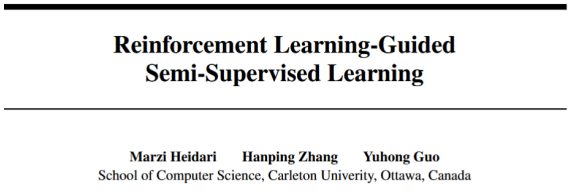
Introduction

- **Problem:** Labeled data is limited and costly to obtain.
- **Challenge:** Traditional pseudo-labeling often propogates errors.
- **Idea:** Use Reinforcement Learning (RL) to adaptively select pseudo-labels and improve error rate.

Background and Literature Review

- The team began with introductory RL implementations using examples from the `gymnasium` library to build foundational understanding.

- Conducted an initial review and implementation of five RL-based pseudo-labeling studies to establish a research baseline.



Reinforcement Learning-Guided Semi-Supervised Learning

Marzi Heidari Hanping Zhang Yuhong Guo
School of Computer Science, Carleton Univerity, Ottawa, Canada



Selective Pseudo-Labeling with Reinforcement Learning for Semi-Supervised Domain Adaptation

Bingyu Liu[1,3], Yuhong Guo[1,2], Jieping Ye[1], Weihong Deng[3]
[1] Didi Chuxing [2] Carleton University [3] Beijing University of Posts and Telecommunications

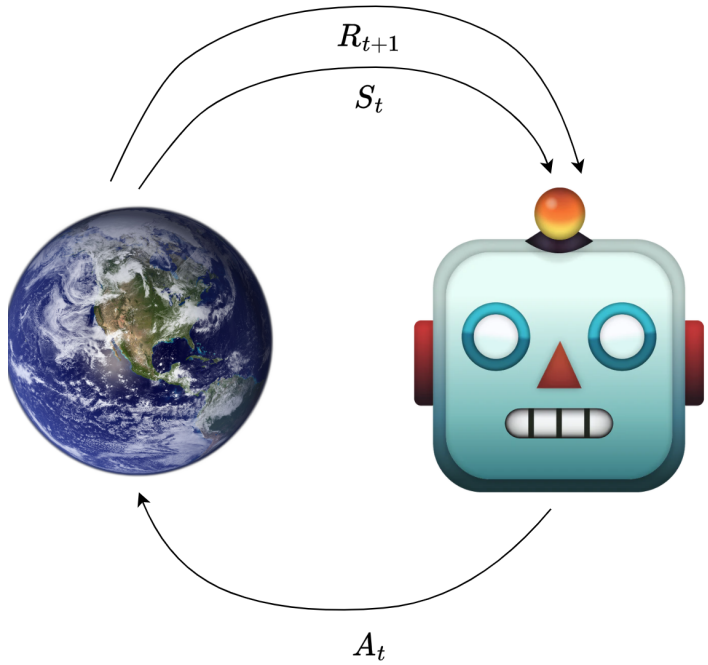


A Review of Pseudo-Labeling for Computer Vision

Patrick Kage* P.KAGE@ED.AC.UK
School of Informatics, The University of Edinburgh, Edinburgh, UK
Jay C. Rothenberger* JAY.C.ROTHENBERGER@OU.EDU
School of Computer Science, The University of Oklahoma, Norman OK, USA
Pavlos Andreadis PAVLOS.ANDREADIS@ED.AC.UK
School of Informatics, The University of Edinburgh, Edinburgh, UK
Dimitrios I. Diochnos DIOCHNOS@OU.EDU
School of Computer Science, The University of Oklahoma, Norman OK, USA

Reinforcement Learning Introduction

- **Reinforcement Learning:** Agent learns through interaction and feedback from the environment.
- **Sequential Decision Making:** Each action affects future states and rewards.
- **Markov Decision Process (MDP):** Framework for modelling decision making.



Markov Decision Process

Approach-1

Reinforcement Learning Using Policy Gradient

MDP Formulation

State

Image features, model predictions, entropy (uncertainty).

$$\mathcal{S} = \{\mathbf{X};\ \text{softmax}(\mathbf{y});\ \mathcal{L}\}$$

Action

Assign pseudo-label or skip.

$$\mathcal{A} = \{0, \ldots, 9, ;\ \text{skip}\}$$

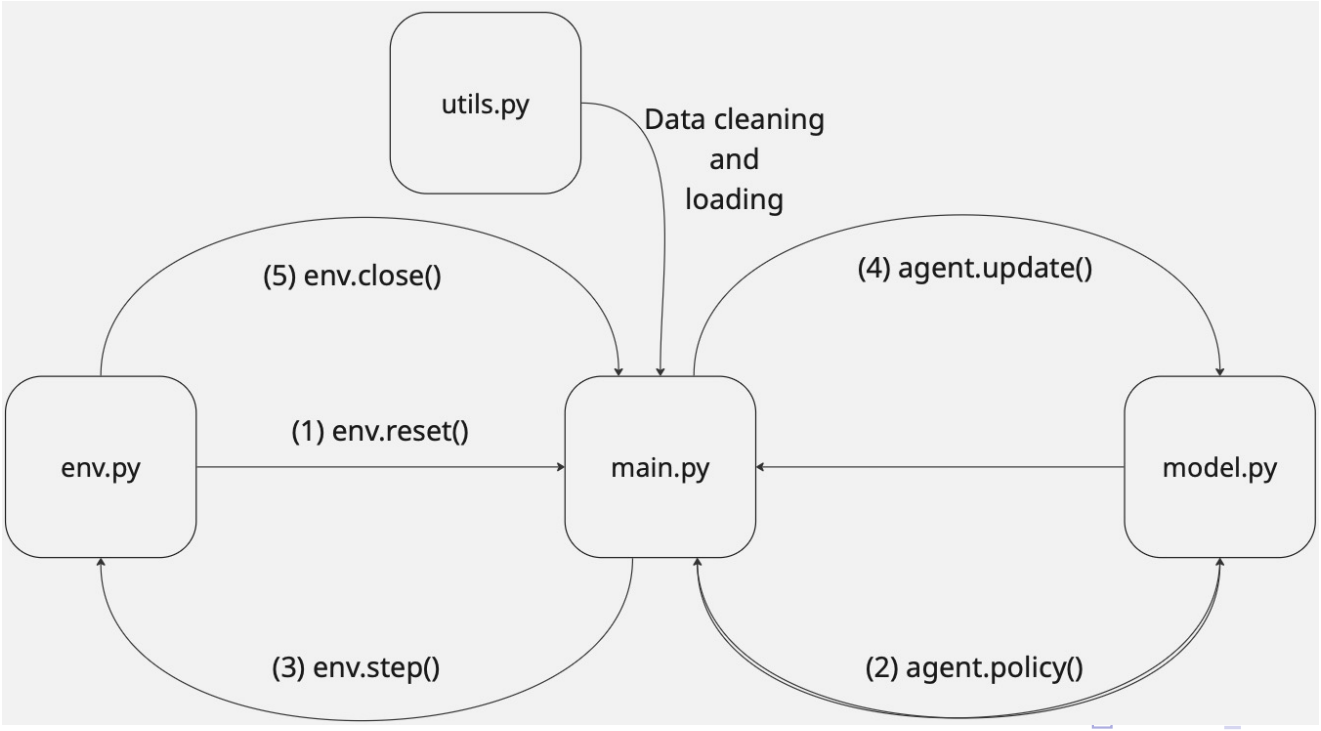
Reward

Error Rate improvement of downstream model on validation data.

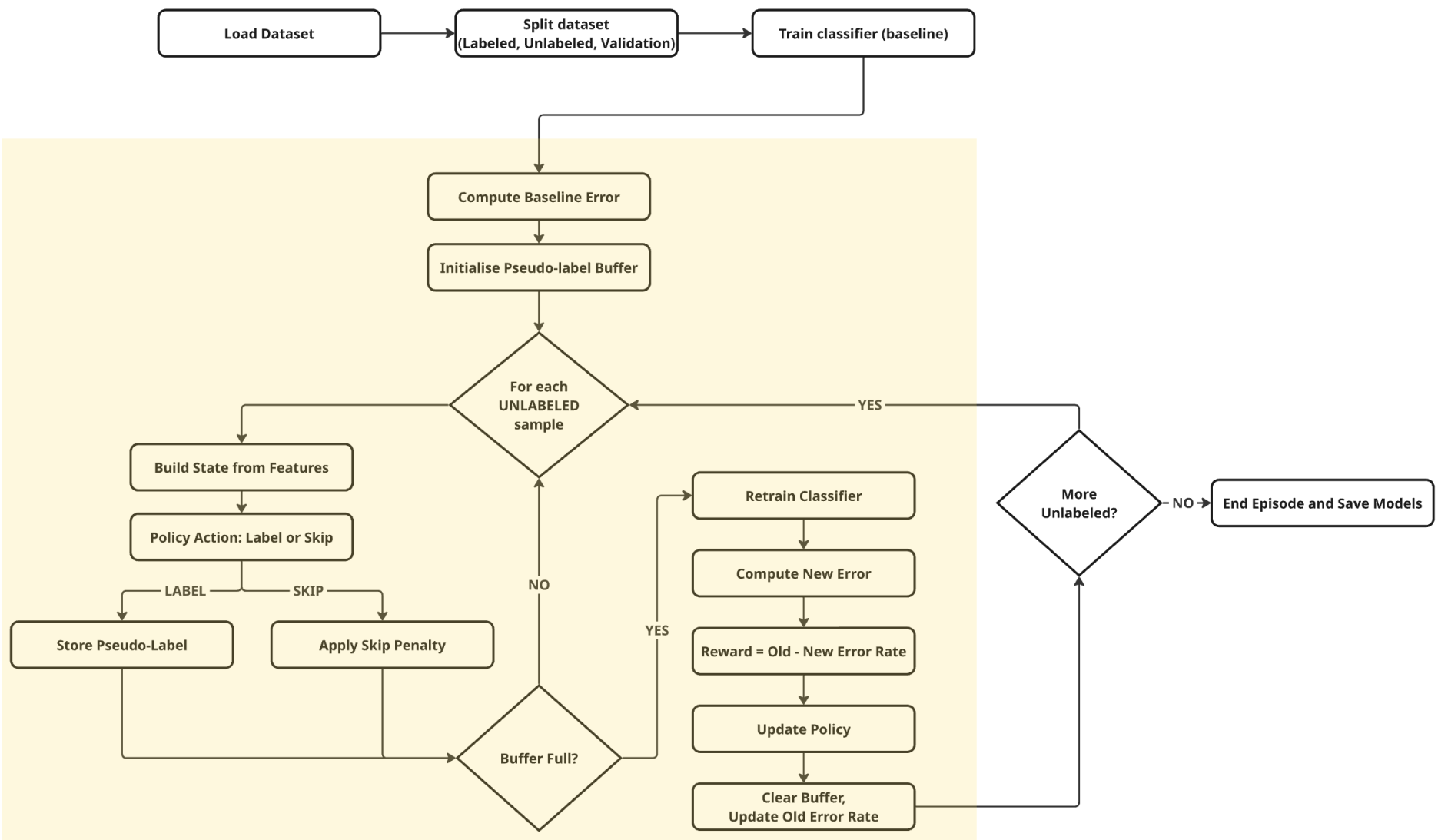
$$\mathcal{R}_t = -(e_t - e_{t-1})$$

Code Files

- **main.py**: Entry point for training process.
- **env.py**: Custom RL environment handling pseudo-labeling episodes.
- **model.py**: Updates policy based on agent's actions for maximising reward.
- **utils.py**: Helper functions for data loading, metrics, etc.



Flow Diagram



PseudoCode

Algorithm RL agent training loop

Require: state_dim, action_dim, labeled_data, unlabeled_data, num_episodes
  agent ← `RLAgent`(state_dim, action_dim)
  env ← `PseudoLabelEnv`(labeled_data, unlabeled_data)

  for episode = 1 **to** num_episodes **do**
    state ← env.reset()
    done ← false
    while not done **do**
      action ← agent.select_action(state)
      (next_state, reward, terminated, info) ← env.step(action)
      state ← next_state
      done ← terminated
    end while
    agent.update()
    error rate ← env.evaluate()
  end for
  env.close()

env.reset()

```
    def reset(self, *, seed=None, options=None):
        super().reset(seed=seed)
        self.current_idx = 0
        self.newly_pseudo_labeled_data = [] # current pseudo-label batch
        self.all_pseudo_labeled_data = [] # accumulated pseudo-label batch
        self.episode_reward = 0.0
        self.current_error_rate = self.baseline_error_rate

        # shuffle unlabeled data
        perm = torch.randperm(self.unlabeled_x.size(0))
        self.unlabeled_x = self.unlabeled_x[perm]
        self.unlabeled_y = self.unlabeled_y[perm]

        # Reset the downstream model to initial baseline state
        self.downstream_model.load_state_dict(self.initial_model_state)
        return self.get_next_state(), {}
```

- **Input:** Random seed and options.
- **Functionality:** Resets the environment.
- **Output:** Initial environment state and empty info dictionary.

model.policy()

```
def policy(self, state):
```

```
    with torch.no_grad():
        state_tensor = torch.FloatTensor(state).unsqueeze(0).to(self.device)
        logits = self.pi(state_tensor)
        action_dist = torch.distributions.Categorical(logits=logits)
        action = action_dist.sample()
        return action.item()
```

- **Input:** Current state (S_t).
- **Functionality:** Action (pseudolabel) is selected based on logits obtained from policy network.

$$a \in \mathcal{A}(s)$$

- **Output:** Selected action as integer.

env.step()

```
    def step(self, action):
        img_idx, reward = self.current_idx, 0.0
        if self.use_true_labels:
            chosen_label = self.unlabeled_y[img_idx]
            self.newly_pseudo_labeled_data.append((self.unlabeled_x[img_idx], chosen_label))
        else: # if running a pseudo labeling experiment
            if action < self.num_classes:
                chosen_label = torch.tensor(action, device=self.device)
                self.newly_pseudo_labeled_data.append((self.unlabeled_x[img_idx], chosen_label))
            else: # skip
                reward += self.calculate_reward(0.0, 0.0, skip_penalty=True)

        if len(self.newly_pseudo_labeled_data) and ((self.current_idx+1 >= len(self.unlabeled_x)) or (len(
    self.newly_pseudo_labeled_data) == 512)):
            old_error_rate = self.current_error_rate
            combined_pseudo_labeled_data = self.all_pseudo_labeled_data + self.newly_pseudo_labeled_data
            self.downstream_model.train_model((self.labeled_x, self.labeled_y),
    combined_pseudo_labeled_data)
            new_accuracy = self.downstream_model.evaluate_model(self.val_x, self.val_y)
            new_error_rate = self.current_error_rate = 1.0 - new_accuracy
            reward += self.calculate_reward(old_error_rate, new_error_rate)
            self.all_pseudo_labeled_data.extend(self.newly_pseudo_labeled_data)
            self.newly_pseudo_labeled_data = [] # Clear the buffer for the next batch

        self.episode_reward, self.current_idx = self.episode_reward+reward, self.current_idx+1
        info, terminated = {}, (self.current_idx >= len(self.unlabeled_x))
        if terminated: info['error_rate'], info['final_reward'] = new_error_rate, self.episode_reward

        return self.get_next_state(), reward, terminated, False, info
```

env.step()

- **Input:** Action selected by the agent.
- **Functionality:** There are multiple functionalities like:
  - Computes reward based on prediction and action.
  - Updates pseudo-labeled data.
- **Output:** Returns next state, total reward, termination flag, and performance info (error rate, final reward).

model.update()

```
1     def update(self, sampled_states, sampled_actions, sampled_rewards):
2
3         # Compute advantages
4         rewards_to_go, values, adv = self.compute_advantage(sampled_states, sampled_rewards)
5
6         # Convert to tensors
7         states_tensor = torch.FloatTensor(np.array(sampled_states)).to(self.device)
8         actions_tensor = torch.LongTensor(sampled_actions).to(self.device)
9
10        # Update policy network
11        self.optimizer_pi.zero_grad()
12        logits = self.pi(states_tensor)
13        log_probs = F.log_softmax(logits, dim=1)
14        action_log_probs = log_probs.gather(1, actions_tensor.unsqueeze(1)).squeeze(1)
15        policy_loss = -torch.mean(action_log_probs * adv.detach())
16        policy_loss.backward()
17        self.optimizer_pi.step()
18
19        # Update value network
20        self.optimizer_v.zero_grad()
21        value_preds = self.v(states_tensor).squeeze(1)
22        value_loss = torch.mean((rewards_to_go - value_preds) ** 2)
23        value_loss.backward()
24        self.optimizer_v.step()
25
26        return policy_loss.item(), value_loss.item()
```

model.update()

- **Input:** Sampled states, actions, and rewards from agent experience.
- **Functionality:** There are multiple functionalities like:
  - Computes advantages using rewards-to-go and value estimates.
  - Updates policy network ($\pi_\theta(a|s)$) to maximize expected reward.
  - Updates value network (V_θ^π) to minimize prediction error.
- **Output:** Returns policy loss and value loss for performance tracking.

main.py (Bringing everything together)

```python
def main():
    writer = SummaryWriter(f"runs/{run_name}")
    env = PseudoLabelEnv(num_labeled, num_unlabeled, num_validation)
    state_dim, action_dim = env.observation_space.shape[0], env.action_space.n
    agent = PolicyGradient(state_dim=state_dim,num_actions=action_dim)
    best_error_rate = float('inf')

    for episode in tqdm(range(episodes), desc="Training Progress"):
        state, info = env.reset()
        sampled_states, sampled_actions, sampled_rewards, terminated = [], [], [], False
        while not terminated:
            action = agent.policy(state)
            next_state, reward, terminated, _, info = env.step(action)
            sampled_states.append(state)
            sampled_actions.append(action)
            sampled_rewards.append(reward)
            state = next_state

        policy_loss, value_loss = agent.update(np.vstack(sampled_states), sampled_actions, sampled_rewards)
        episode_reward = sum(sampled_rewards)  # Sum of step rewards
        if 'final_reward' in info:
            final_reward, error_rate = info['final_reward'], info['error_rate']
            if error_rate < best_error_rate:
                best_error_rate = error_rate
                env.downstream_model.save_model(name='best_downstream', save_dir=save_dir)
                agent.save_model(name='best_policy_gradient', save_dir=save_dir)

    env.close()
    writer.close()
```

Key Observations

- CNN benefits more from RL guidance than ResNet due to weaker baseline.
- Performance degradation observed after multiple episodes.
- **Key Bottleneck:** Reward design and confirmation bias.

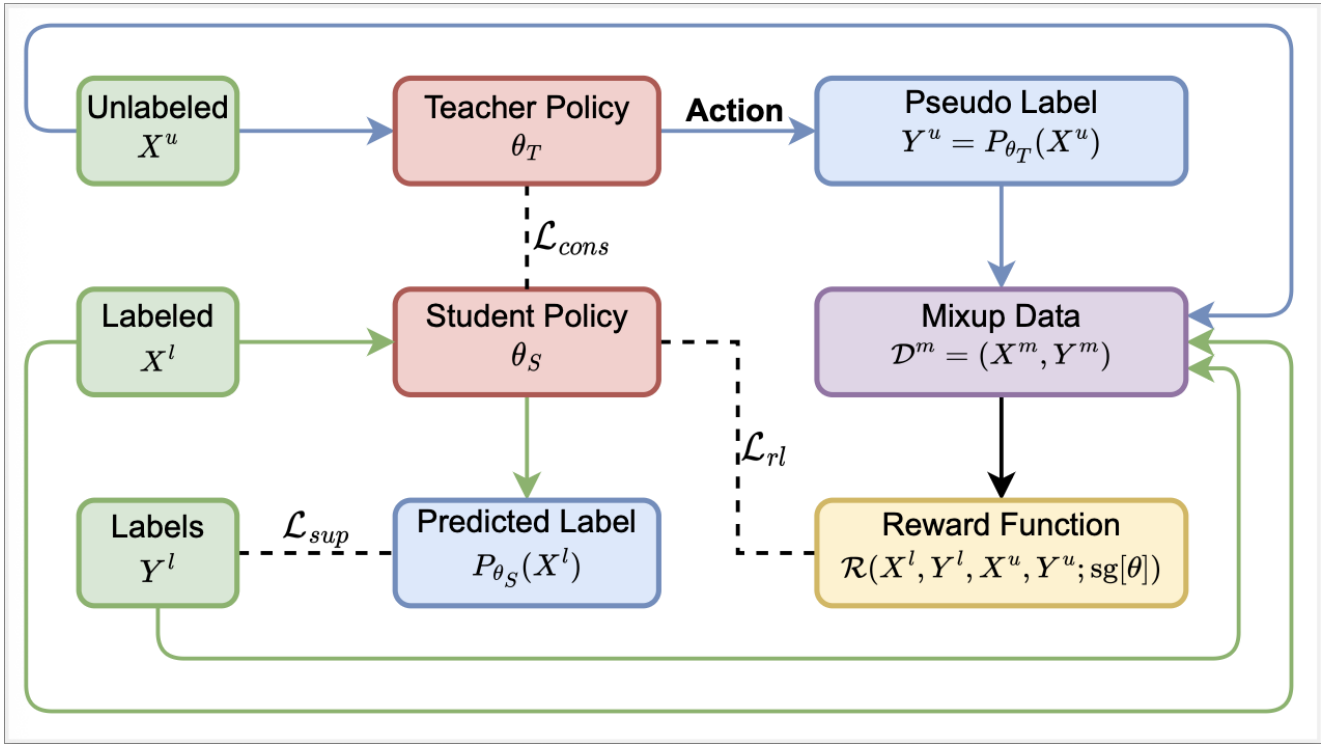
Model	**MNIST**	**CIFAR-10**
CNN	3.45%	34.40%
MLP	37.40%	59.60%
ResNet18	5.80%	25.75%

Table: Performance comparison on MNIST and CIFAR-10

Final Classification Error rate Comparison

Approach-2

Reinforcement Learning Guided Semi-Supervised Learning (RLGSSL)



PseudoCode RLGSSL

Algorithm RL-Guided SSL

Require: $\mathcal{D}^l$, $\mathcal{D}^u$, and extended $\widetilde{\mathcal{D}}^l$; initialized θ_S, θ_T; hyperparameters
  for iteration $= 1$ to maxiters **do**
    for each $x_i^u \in \mathcal{D}^u$ **do**
      Compute soft pseudo-label vector $y_i^u = P_{\theta_T}(x_i^u)$ to form (x_i^u, y_i^u)
    end for
    Generate mixup data $\mathcal{D}^m = (X^m, Y^m)$ on $\mathcal{D}^u$ and $\widetilde{\mathcal{D}}^l$ using with shuffling
    for step $= 1$ to maxsteps **do**
      Draw a batch of data $B = \{(x_i^m, y_i^m)\}$ from $\mathcal{D}^m$
      Calculate the reward function $\mathcal{R}(\cdot; sg[\theta_S])$ using the batch B
      Compute the objective
      Update the policy parameters θ_S via gradient descent
      Update teacher model θ_T via EMA
    end for
  end for

Key Observations RLGSSL

- Performance is affected by too many hyperparameters.
- **Key Bottleneck:** Architectural Complexity.

Epoch	Error (%)	Loss	RL Loss	Reward
50 (warmup)	53.61	0.137	-0.00533	-0.0305168
100	74.02	0.233	0.018	-0.0298085
150	66.56	0.215	0.013	-0.029022
250	59.48	0.188	0.005	-0.0290487
350	61.51	0.1601	-0.00464	-0.0283258
450	58.59	0.13346	-0.03042	-0.0281552

Table: RLGSSL training progression on CIFAR-10 (1000 labels).

Updates from Mid-Term

- Built a model and data agnostic pseudo-labeling system
- Updated and refactored core environment methods
- Excluded true labels from the training process
- Experimented with multiple loss functions from the RLGSSL paper
- Developed a custom environment that is ready to be registered with OpenAI Gym

Conclusion

- Successfully demonstrated RL-based pseudo-labeling in semi-supervised learning.
- Long-term training stability remains a critical challenge.
- Reward design plays a central role in RL-based pseudo-labeling.
- The framework shows strong research potential but needs optimization.

Limitations

- **Performance Degradation**: Error rate increases after several episodes due to confirmation bias.
- **Reward Sensitivity**: Small reward changes cause unstable training.
- **Computational Cost**: RL training is 3–4× slower than standard pseudo-labeling.
- **Noisy Labels**: Early incorrect pseudo-labels propagate errors.

Future Scope

- Implement Proximal Policy Optimization (PPO) for improved stability.
- Experiment with diverse reward strategies.
- Extend to domain adaptation and real-world noisy datasets.
- Publish the custom PseudoLabelEnv on OpenAI Gym

Thank You

Questions?